

picoLLM: A Compact Study of Small-Scale Language Modeling

Thang Sian Kop
New York University
New York, NY, USA
ts5783@nyu.edu

Abstract—This paper presents *picoLLM*, a compact language modeling project designed to study how small neural language models behave under simple and computationally lightweight settings. Instead of relying on large-scale corpora and expensive training pipelines, *picoLLM* focuses on short text sequences, controlled experiments, and reproducible comparisons across model variants. The project supports training, experiment tracking, result collection, and visualization, making it useful both as a learning platform and as an experimental framework for studying core language modeling ideas. A comparison across three K-gram model variants—Embedding, Conv, and Onehot—shows that the Embedding variant achieved the best overall trade-off between predictive quality and efficiency in the reported experiments. This project demonstrates that small-scale language modeling can still produce meaningful insight into representation choice, optimization behavior, and performance trade-offs.

I. INTRODUCTION

Large language models have become central to modern natural language processing, but they are often difficult to study closely because they require large datasets, substantial compute, and complex infrastructure. For students and early researchers, this makes it harder to build intuition about how language modeling works at a fundamental level.

picoLLM was developed as a compact experimental environment for studying core language modeling ideas in a more accessible setting. Instead of training on massive internet-scale text, the project uses smaller datasets and simpler experimental pipelines. This makes it easier to test how changes in model structure and training settings affect performance.

The main contribution of this project is not scale, but clarity. By reducing the size and complexity of the system, *picoLLM* makes it easier to observe the effects of representation choices, compare multiple runs, and interpret model behavior in a more controlled way.

II. PROJECT DESIGN

The project is centered on next-token prediction. Given a short context, the model predicts the next token in the sequence. This setup captures the core learning problem behind autoregressive language modeling while remaining simple enough for controlled experimentation.

The *picoLLM* repository includes several components that support this workflow:

- `pico-llm.py` as the main script for training and testing,

- `pico-llm.ipynb` for notebook-based experimentation,
- a `data/` directory for small training corpora,
- an `artifacts/` directory for saved models, logs, and plots,
- and helper scripts for collecting experiments and generating plots.

This structure allows the project to function as both a coding project and a compact experimental framework.

III. METHODOLOGY

The project studies small-scale language modeling by comparing different K-gram model variants. The goal is to understand how representation and architecture affect both learning quality and computational efficiency.

For a context window of size k , the model learns a conditional distribution of the form

$$P(x_t \mid x_{t-k}, \dots, x_{t-1}), \quad (1)$$

where x_t is the next token to be predicted. Training minimizes cross-entropy loss over the predicted token distribution.

The reported comparison in the repository evaluates three variants:

- Embedding
- Conv
- Onehot

These variants provide a simple but useful basis for understanding how different modeling choices affect loss and throughput.

IV. EXPERIMENTAL RESULTS

The repository reports two main metrics for the K-gram comparison:

- average loss, where lower is better, and
- throughput in tokens per second, where higher is better.

Table I summarizes the values shown in the repository figure.

The results suggest that the Embedding model provided the best overall trade-off in the reported experiments. It achieved the lowest loss, which indicates stronger predictive performance, and also the highest throughput, which indicates better computational efficiency. The Conv model performed in

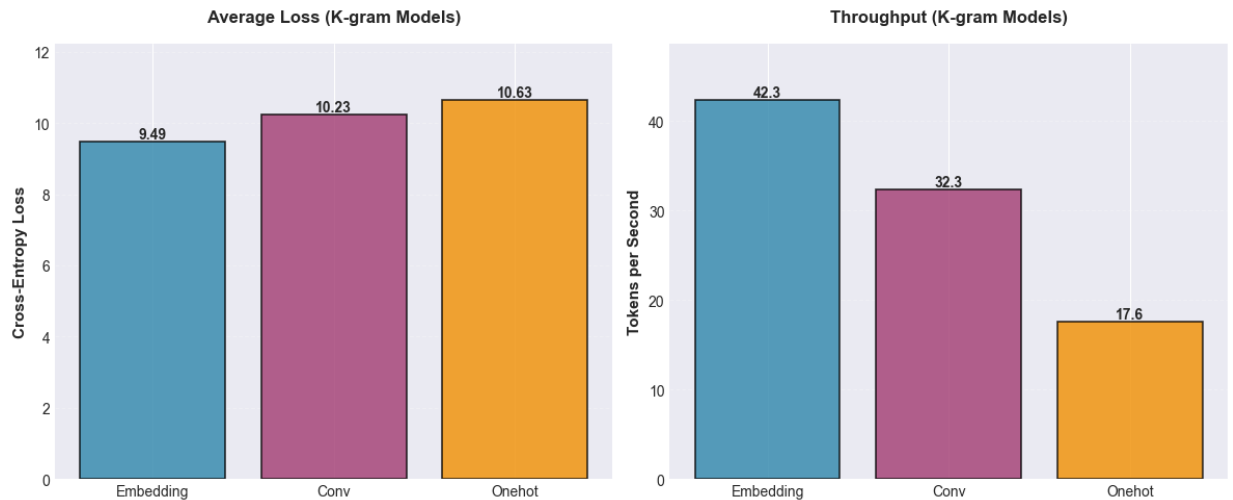


Fig. 1. K-gram model comparison from picoLLM. The left chart shows average loss, and the right chart shows throughput in tokens per second. The Embedding variant achieved both the lowest loss and the highest throughput among the three tested variants.

TABLE I
K-GRAM COMPARISON IN PICO LLM

Variant	Average Loss	Tokens / Second
Embedding	9.49	42.3
Conv	10.23	32.3
Onehot	10.63	17.6

the middle, while the Onehot model had both the highest loss and the slowest runtime.

These findings are useful because they show that even in a compact language modeling project, representation choice matters. A better representation can improve both predictive quality and speed.

V. DISCUSSION

One strength of picoLLM is that it makes experimentation manageable. Because the project is small, multiple runs can be compared more easily, and the effect of changing one variable at a time becomes more visible. This supports a more research-oriented way of working, where the goal is not only to train a model, but also to understand why certain choices perform better than others.

The project also has educational value. Building and comparing compact models helps develop intuition about token prediction, cross-entropy loss, efficiency trade-offs, and experiment design. These are foundational ideas that carry over to larger language modeling systems.

At the same time, the project shows that small systems can still produce meaningful technical observations. Even without massive infrastructure, compact experiments can reveal useful patterns about model behavior.

VI. LIMITATIONS

This project has several limitations. First, the scale is intentionally small, so the results should not be interpreted as

claims about large production-scale language models. Second, the evaluation is focused on compact experimental metrics such as loss and throughput, rather than broader capabilities like instruction following or long-context reasoning. Third, the project is simplified by design, which improves interpretability but limits direct comparison with modern large-scale systems.

VII. CONCLUSION

This paper presented picoLLM as a compact framework for studying language modeling in a simple and reproducible setting. By focusing on small datasets, manageable experiments, and structured comparisons, the project makes it easier to understand how model choices affect both predictive quality and efficiency. The K-gram comparison showed that the Embedding variant outperformed the Conv and Onehot variants in the reported experiments, achieving both lower loss and higher throughput. Overall, picoLLM demonstrates that small-scale language modeling can still be a valuable tool for technical learning, experimentation, and early-stage research.

PROJECT REPOSITORY

<https://github.com/siankop22/picoLLM>